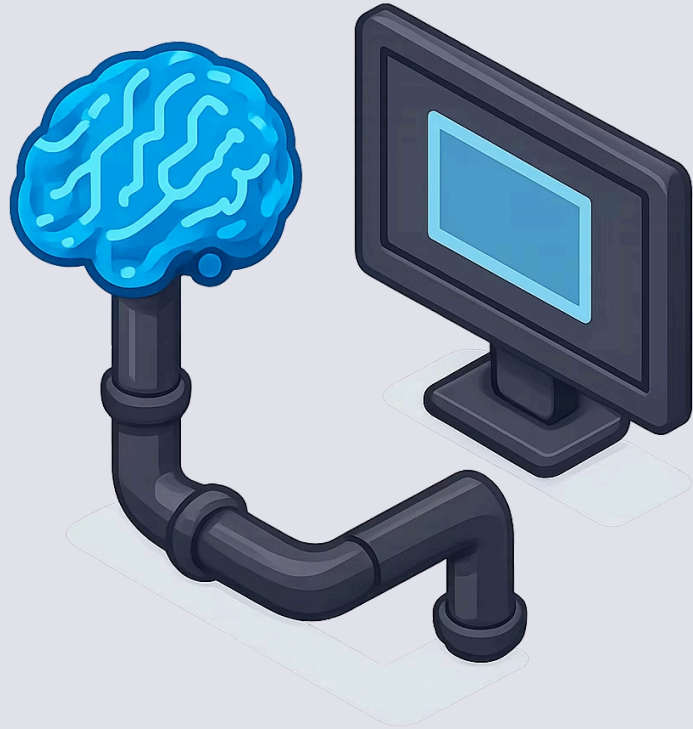




LangChain — From Basics to Advanced

A comprehensive guide to building intelligent AI applications using LangChain — covering concepts, architecture, code, and real-world applications.

Dr. Anilkumar Vishwanath Brahmane

Department of Computer Engineering

Sanjivani College of Engineering

What is LangChain?

Simple LLM

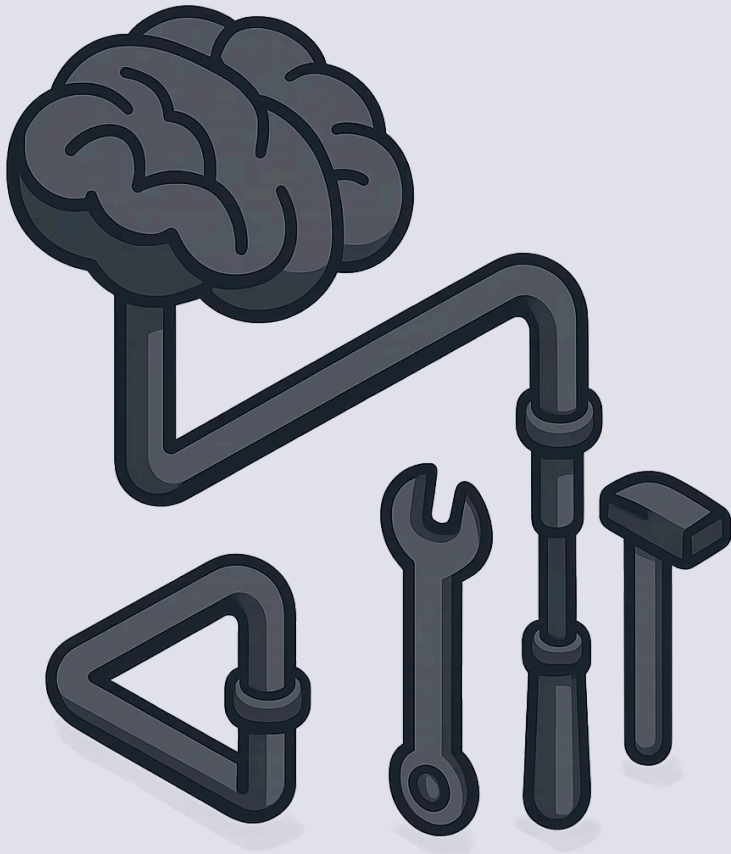
User
↓
Question
↓
LLM
↓
Answer

Works for simple questions only.

LangChain Pipeline

User → Question
↓
Understand question
↓
Search documents
↓
Retrieve information
↓
Send to LLM
↓
Use tools if needed
↓
Remember conversation
↓
Final Answer

- 📄 **LangChain** is a Python framework that connects an LLM with prompts, documents, databases, tools, and memory to build intelligent AI applications.
LLM = Brain. LangChain = System that connects the brain with tools and information.



LangChain vs. LLM — The Restaurant Analogy

 Restaurant = AI Application

The complete system that serves the customer end-to-end.

 Chef = LLM

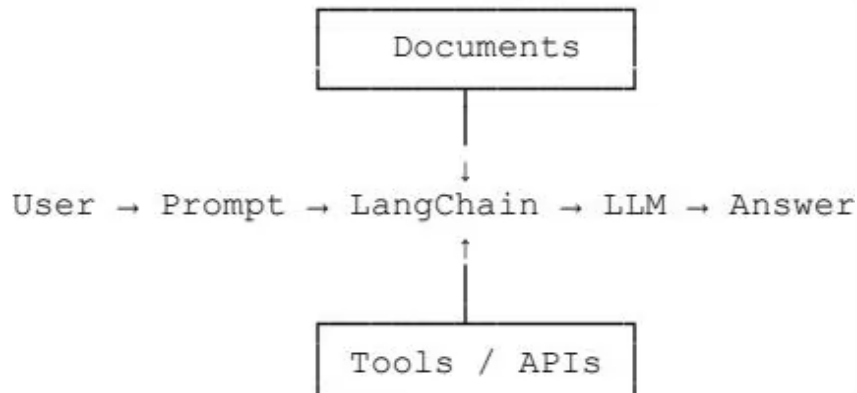
The brain that generates the final answer (food). GPT, Gemini, Claude.

 Waiter = LangChain

Coordinates orders, ingredients, and tools — connects everything together.

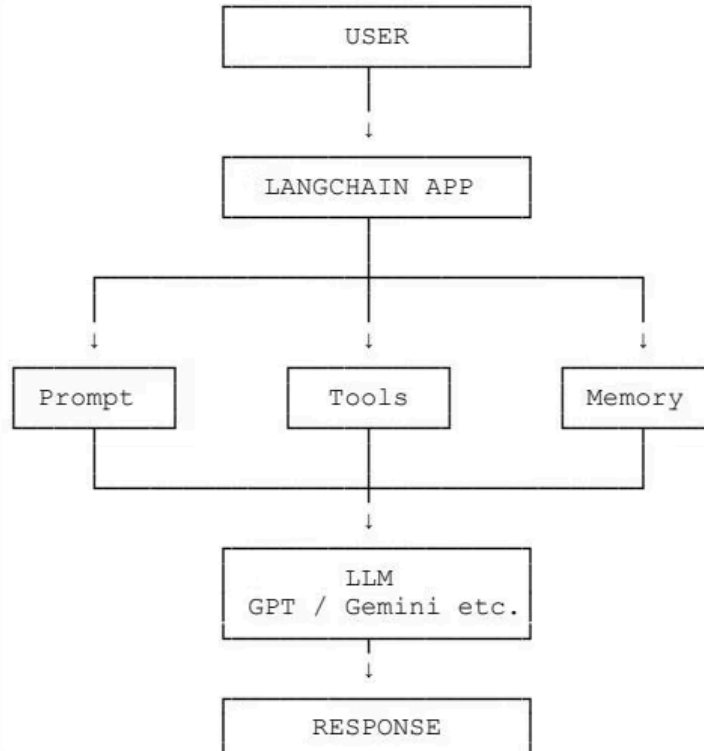
 Kitchen Tools = APIs/Tools

Calculators, databases, search engines — external capabilities.



Basic LangChain Architecture

Architecture Flow



Main Building Blocks

Prompt

Instructions given to the LLM.

Tools

External APIs, calculators, databases.

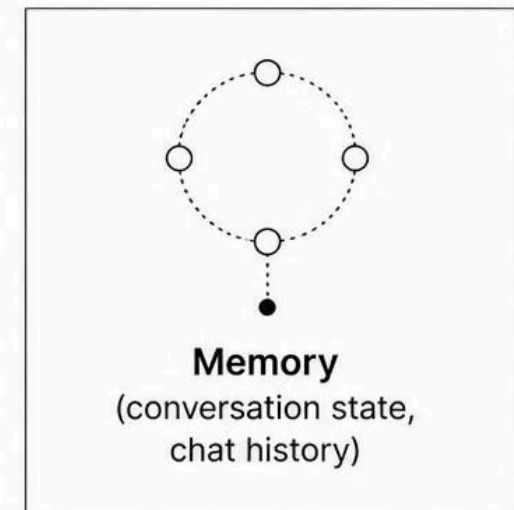
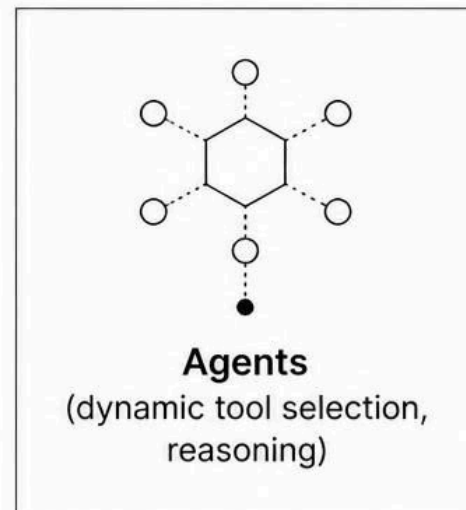
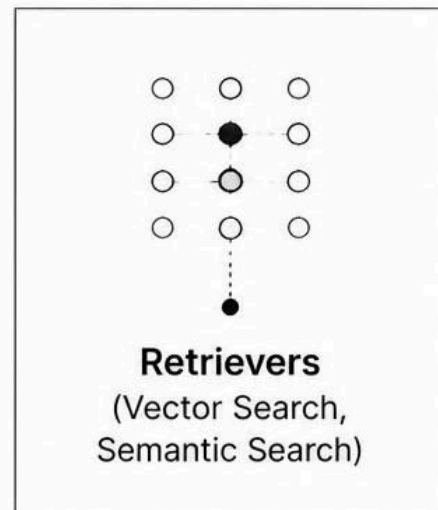
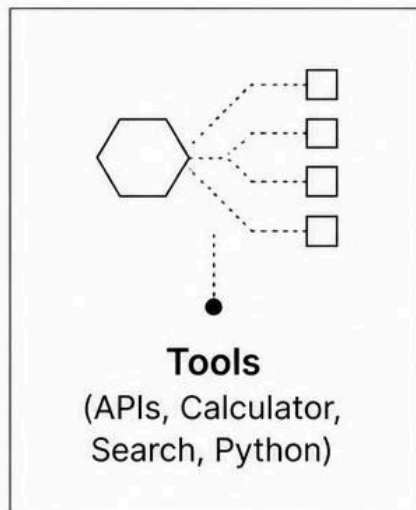
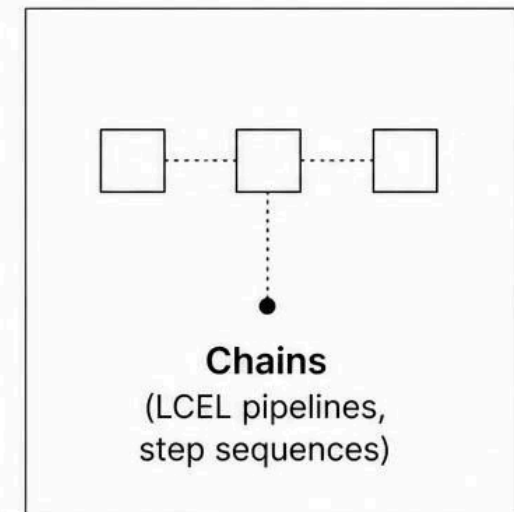
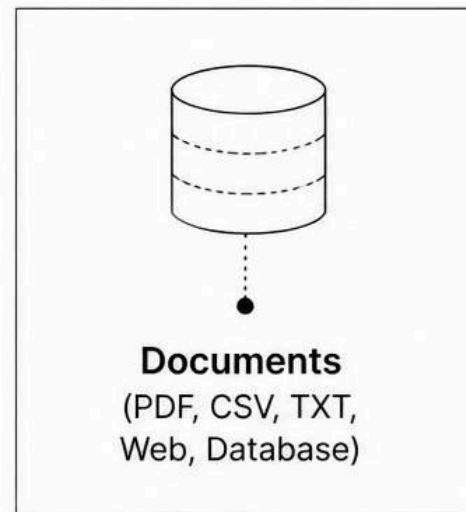
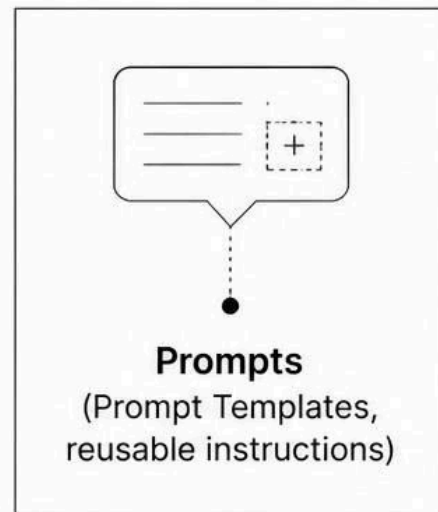
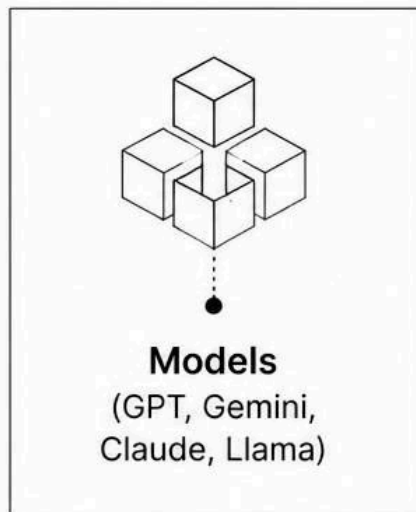
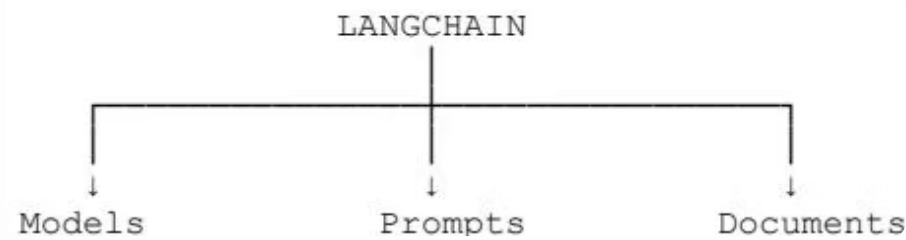
Memory

Maintains conversation/workflow state.

LLM

GPT / Gemini / Claude – the AI brain.

Core Components of LangChain



LangChain is divided into these key building blocks. **Models** are the AI brains. **Prompts** guide them. **Chains** connect steps. **RAG** adds external knowledge. **Agents** decide dynamically what to do.

Chains, LCEL & Output Parsers

What is a Chain?

A chain connects multiple steps together in a pipeline.

Question

↓ Prompt

↓ LLM

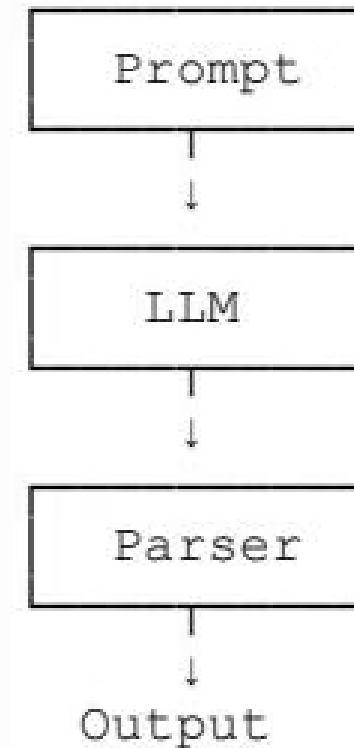
↓ Answer

LCEL — LangChain Expression Language

```
chain = prompt | llm | parser
```

The | symbol pipes output of one step into the next — like a Unix pipeline.

Complete LCEL Pipeline



Output Parser

LLMs return text. An **Output Parser** converts that text into structured data your application can use — e.g., JSON objects with name, age, department fields.

Your First LangChain Python Program

Concept Flow

```
graph TD; A[User Question] --> B[Prompt Template]; B --> C[Gemini LLM]; C --> D[Output Parser]; D --> E[Clean Answer];
```

Install

```
pip install langchain langchain-google-genai
```

Python Code

```
# Step 1: Import Gemini integration
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# Step 2: Create the LLM (AI Brain)
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash"
)

# Step 3: Create a reusable prompt template
prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in simple English for beginners."
)

# Step 4: Create output parser (clean text)
parser = StrOutputParser()

# Step 5: Build the chain using LCEL pipe symbol
chain = prompt | llm | parser

# Step 6: Run the chain with a topic
answer = chain.invoke({"topic": "Machine Learning"})

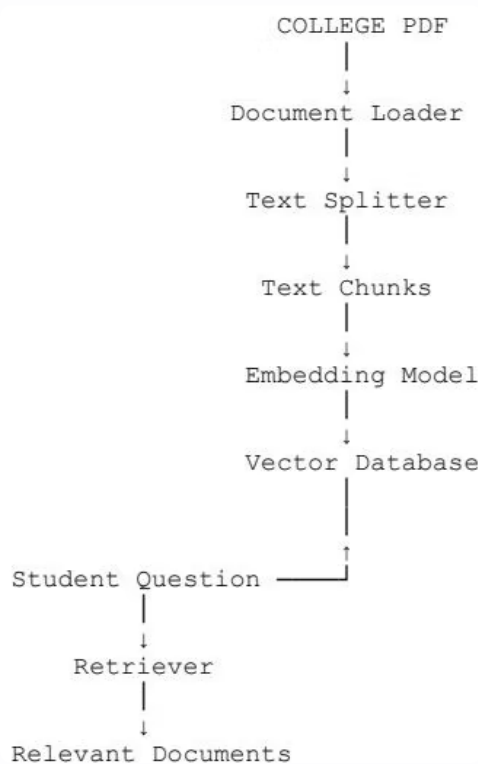
# Step 7: Display the result
print(answer)
```

📌 The `|` symbol connects Prompt → LLM → Parser into one pipeline. **invoke()** sends input through the entire chain and returns the final answer.

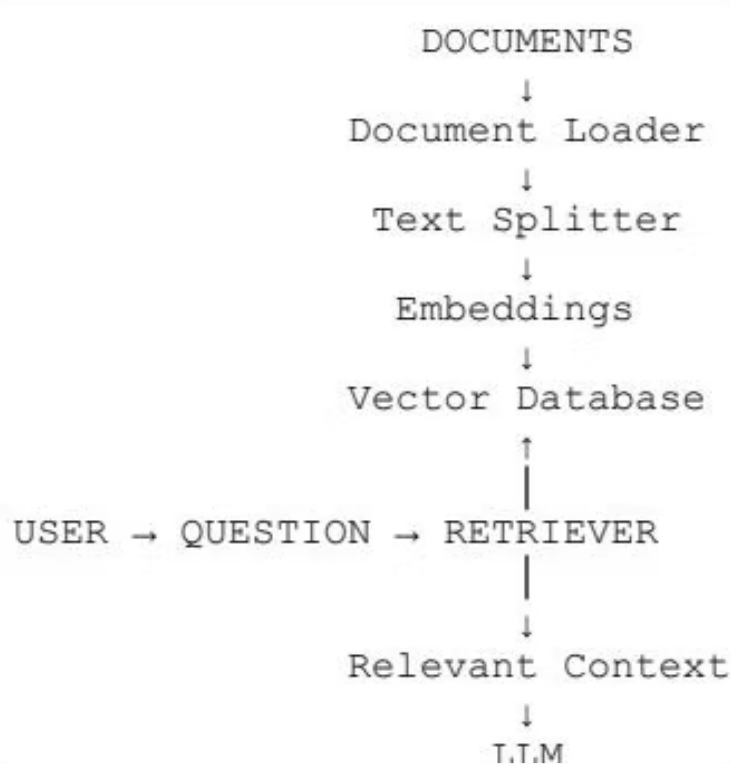
RAG — Retrieval Augmented Generation

RAG = First retrieve useful information, then ask the LLM to generate an answer using that information. This solves the problem of LLMs not knowing your specific documents.

Indexing Phase (One-time)



Query Phase (Every question)



RAG — Documents, Embeddings & Vector Databases

Document Processing Pipeline

PDF / Word / CSV / Website
↓ Document Loader
↓ Text Splitter
↓ Chunks (smaller pieces)
↓ Embedding Model
↓ [0.12, -0.43, 0.78, ...]
↓ Vector Database
(FAISS / Chroma / Pinecone)

Why Chunks?

A 100-page PDF cannot be sent to the LLM every time. We split it into smaller **chunks** and only retrieve the relevant ones.

RAG Python Structure

```
# 1. Load document
documents = loader.load()

# 2. Split into chunks
chunks = splitter.split_documents(documents)

# 3. Create embeddings (text → numbers)
embeddings = embedding_model.embed(chunks)

# 4. Store in vector database
vector_store = Chroma.from_documents(chunks)

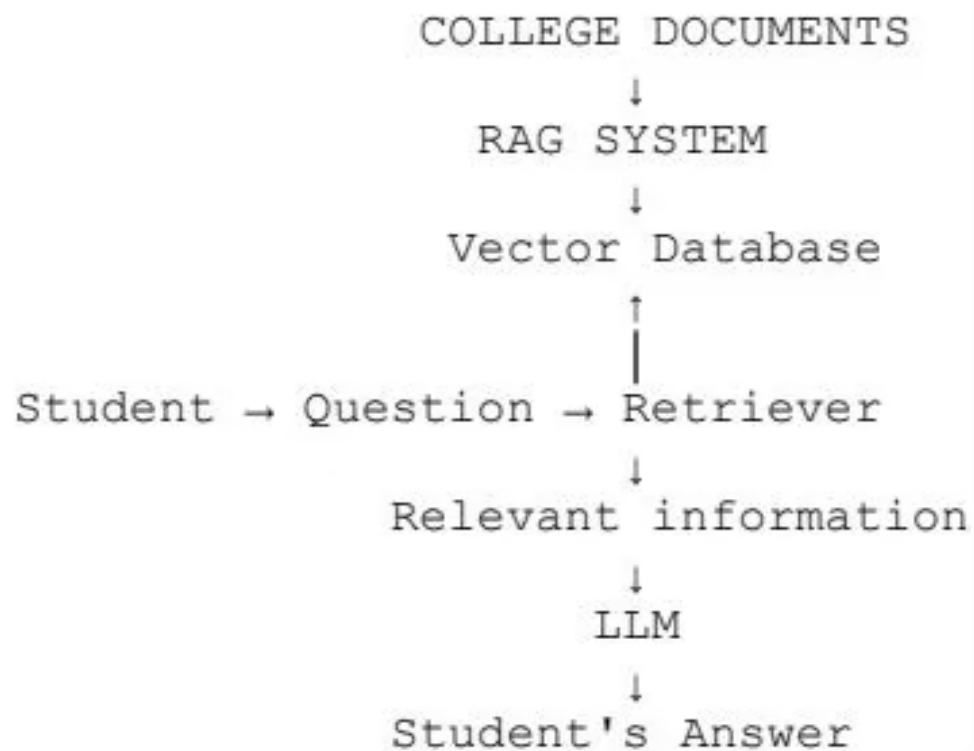
# 5. Create retriever
retriever = vector_store.as_retriever()

# 6. Retrieve relevant chunks for question
docs = retriever.invoke("What is attendance rule?")

# 7. Send context + question to LLM
answer = llm.invoke(context + question)

print(answer)
```

College RAG System — Real Example



College Documents

Academic Regulations, Examination Rules, Attendance Rules, Course Syllabus PDFs.

RAG System

Loads, splits, embeds documents into a Vector Database for semantic search.

Student Question

"How much attendance is required?" → Retriever finds relevant rule → LLM answers.

Accurate Answer

Much more reliable than relying only on the LLM's general training knowledge.

Tools & Agents — LangChain's Superpower

Why Tools?

LLMs alone cannot reliably calculate, fetch live data, or query databases.

Tools give the LLM external capabilities.



Calculator

Reliable math: 12567×876



Weather API

Live data the LLM doesn't know.



Database

Query company-specific records.

Chain vs. Agent

CHAIN (predefined sequence):

$A \rightarrow B \rightarrow C \rightarrow D$

Steps are fixed at design time.

AGENT (dynamic decisions):

User $\rightarrow$ Agent

$\rightarrow$ Weather Tool

$\rightarrow$ Calculator Tool

$\rightarrow$ Database

$\rightarrow$ LLM $\rightarrow$ Answer

Agent decides WHICH tool to use
and in WHAT order — at runtime.

```
User Question
  ↓
  Agent
  ↓
Need weather information
  ↓
Weather Tool
  ↓
Temperature
  ↓
Need conversion
  ↓
Calculator Tool
  ↓
Final Answer
```

Industry Applications of LangChain



E-Commerce Support

AI agent checks order database, company policy, and generates a personalized response to "My order hasn't arrived."



Banking Assistant

Retrieves bank policy via RAG, queries account API, and answers "What documents do I need for a home loan?"



Healthcare

Searches hospital guidelines and medical protocols to assist doctors — as a **decision-support tool** with clinical safeguards.



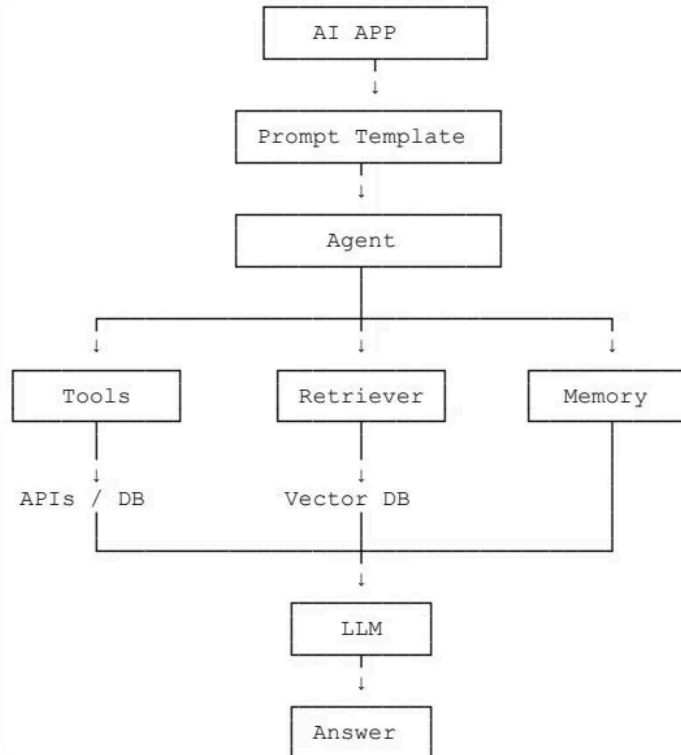
AI Teaching Assistant

Searches syllabus, lecture notes, and lab manuals to explain topics, generate MCQs, viva questions, and personalized summaries.

Dr. Anilkumar Vishwanath Brahmane / Dept. of Computer Engineering / Sanjivani College of Engineering

Advanced Architecture & Security

Complete LangChain App



⚠ Security Considerations

LangChain does **not** automatically make an application secure. Developers must implement:

- Authentication & Authorization
- Data privacy & API security
- Prompt injection protection
- Tool permissions (limited access)
- Human approval for high-risk actions
- Hallucination & data leakage controls

Agent

- ↓ Permission **Check**
- ↓ Allowed Tool **Only**
- ↓ Limited Data

LangChain Learning Roadmap & LangGraph

Beginner → Advanced Path



Level 1–3: Foundations

LLM basics → Prompt Templates → Chains → LCEL → Output Parsers → Document Loaders → Text Splitters



Level 4–5: Knowledge & Tools

Embeddings → Vector Databases → RAG → Retrievers → Tools & APIs



Level 6–7: Agentic AI

Agents → Agentic Systems → LangGraph → Production AI Systems

LangChain vs. LangGraph

LangChain

Building blocks: LLM, Prompt, RAG, Tools, Chains, Agents, Integrations.

LangGraph

Complex stateful workflows: multi-agent loops, human approval steps, dynamic branching.

📌 **LangChain = Building blocks. LangGraph = Controlling complex workflows and agent state.**

The Big Picture — Key Takeaways

LLM

AI brain that understands and generates language. (GPT, Gemini, Claude)

LangChain

Framework connecting LLMs with prompts, data, tools, and workflows.

RAG

Retrieve external information first, then generate a grounded answer.

Agents

AI that dynamically decides which tools and actions to use at runtime.

The simplest sentence to remember: LangChain provides building blocks for connecting LLMs with prompts, data, retrieval systems, tools, and workflows so that we can build real-world Generative AI and Agentic AI applications.

LLM → Prompt → Chain → RAG → Tools → Agents → Agentic AI

